

MARKETPLACE UNIVERSITARIO

UNIMARKET

Documentacion del sistema movil

Presentado por: Mendoza, Alejandra

Carrera: Ingenieria en Sistemas

Asignatura: Proyecto final

Ciudad: Guayaquil

Fecha: 18 de junio de 2026

Resumen

El presente documento describe el desarrollo de UniMarket, una aplicación móvil Android orientada a facilitar la compra, venta, publicación y comunicación entre estudiantes universitarios. El sistema fue implementado en Kotlin con Material Design 3, navegación mediante una sola Activity principal y varios Fragments para representar las interfaces de login, registro, inicio, detalle de producto, publicación, favoritos, chat y perfil. En esta primera versión se utiliza un repositorio local en memoria para simular usuarios, productos, favoritos y mensajes. Adicionalmente, se preparó un script de base de datos PostgreSQL con tablas para usuarios, categorías, productos, multimedia, favoritos, conversaciones y mensajes, con el objetivo de conectar posteriormente la aplicación mediante una API REST segura.

Indice de contenidos

- 1. Introducción
 - 1.1 Justificación
 - 1.2 Planteamiento del trabajo
 - 1.3 Estructura de la memoria
- 2. Contexto y estado del arte
- 3. Objetivos concretos y metodología de trabajo
- 4. Desarrollo específico de la contribución
- 5. Conclusiones y trabajo futuro
- 6. Bibliografía
- Anexos

Indice de tablas

- Tabla 1. Tecnologías utilizadas
- Tabla 2. Requisitos funcionales principales
- Tabla 3. Interfaces del sistema
- Tabla 4. Modelo de datos principal
- Tabla 5. Eventos principales de la aplicación

Indice de figuras

- Figura 1. Arquitectura general propuesta para UniMarket

1. Introduccion

UniMarket es una aplicacion movil de tipo marketplace universitario. Su finalidad es permitir que los estudiantes publiquen productos o servicios academicos, consulten publicaciones disponibles, filtren por categoria, guarden favoritos y contacten a vendedores mediante un chat interno. El desarrollo se realizo en Android Studio usando Kotlin, XML layouts, ViewBinding, Material Design 3 y Navigation Component.

La aplicacion se encuentra en una primera etapa funcional sin backend conectado. Para esta fase se implemento un repositorio local llamado MarketplaceRepository que almacena datos simulados en memoria. Esto permite probar la navegacion, las interfaces y los eventos antes de conectar la persistencia real con PostgreSQL.

1.1 Justificacion

En el entorno universitario es comun que los estudiantes necesiten comprar o vender libros, calculadoras, materiales de laboratorio, accesorios, servicios de tutoria u otros recursos academicos. Sin embargo, esta actividad suele realizarse mediante grupos informales de mensajeria o redes sociales, lo que dificulta la busqueda por categoria, el seguimiento de publicaciones y la comunicacion ordenada entre comprador y vendedor.

UniMarket responde a esta necesidad mediante una solucion movil centrada en la comunidad universitaria. El sistema organiza las publicaciones por categorias, permite gestionar favoritos y ofrece una base para integrar mensajeria, autenticacion y almacenamiento permanente.

1.2 Planteamiento del trabajo

El trabajo propone el desarrollo de una aplicacion Android que funcione como prototipo operativo de un marketplace universitario. La solucion actual incluye las pantallas principales, flujo de navegacion, manejo de eventos y datos locales de prueba. Para una segunda fase se plantea la conexion de la aplicacion con una API REST y una base de datos PostgreSQL.

1.3 Estructura de la memoria

La memoria se organiza en seis capitulos. El primero presenta la introduccion y justificacion. El segundo describe el contexto tecnologico. El tercero establece objetivos y metodologia. El cuarto detalla el desarrollo de la aplicacion. El quinto presenta conclusiones y trabajo futuro. Finalmente, se incluye una bibliografia y anexos tecnicos con los archivos principales del proyecto.

2. Contexto y estado del arte

Las aplicaciones de marketplace permiten publicar productos, consultar catalogos, aplicar filtros, guardar elementos de interes y comunicar compradores con vendedores. En el caso universitario, estas funciones deben adaptarse a un contexto mas cerrado, donde los usuarios pertenecen a una institucion y los productos estan relacionados con necesidades academicas o de vida estudiantil.

Para el desarrollo móvil se utilizó Android Studio con Kotlin. La interfaz se construyó mediante archivos XML y componentes Material Design 3. La navegación se implementó con Navigation Component, usando una sola Activity principal y diferentes Fragments. Esta organización facilita separar las pantallas sin crear una Activity para cada vista.

Figura 1. Arquitectura general propuesta para UniMarket

Android App (MainActivity + Fragments) -> API REST futura -> PostgreSQL

3. Objetivos concretos y metodología de trabajo

3.1 Objetivo general

Desarrollar una aplicación móvil Android para un marketplace universitario que permita registrar usuarios, gestionar publicaciones, buscar productos por categoría, administrar favoritos y facilitar la comunicación entre compradores y vendedores.

3.2 Objetivos específicos

- Permitir el registro e inicio de sesión de usuarios.
- Gestionar publicaciones de productos mediante creación, edición, eliminación y marcado como vendido.
- Facilitar la búsqueda y filtrado de productos por categoría.
- Permitir la comunicación entre comprador y vendedor mediante una pantalla de chat.
- Gestionar una lista de productos favoritos por usuario.
- Preparar la estructura de base de datos PostgreSQL para una futura integración con API REST.

3.3 Metodología del trabajo

1. Análisis de requisitos a partir de las funcionalidades solicitadas para el marketplace.
2. Diseño de interfaces usando XML y componentes Material Design 3.
3. Implementación de navegación con una Activity principal y Fragments.
4. Creación de un repositorio local en memoria para simular datos mientras no existe backend.
5. Elaboración del script PostgreSQL con las tablas necesarias para la persistencia futura.
6. Verificación mediante compilación del APK debug y ejecución de pruebas unitarias locales.

4. Desarrollo específico de la contribucion

4.1 Tecnologías utilizadas

Capa	Tecnología	Uso en el sistema
Frontend movil	Android Studio + Kotlin	Implementacion principal de la aplicacion Android.
Interfaz	XML layouts + Material Design 3	Diseño visual de formularios, botones, tarjetas, chips y pantallas.
Navegacion	Navigation Component	Cambio entre Fragments desde una sola Activity principal.
Binding	ViewBinding	Acceso seguro a vistas declaradas en XML.
Backend futuro	API REST	Comunicacion futura entre Android y la base de datos.
Base de datos	PostgreSQL	Persistencia futura de usuarios, productos, multimedia, favoritos y mensajes.

Tabla 1. Tecnologías utilizadas

4.2 Requisitos funcionales principales

Modulo	Requisito	Estado actual
Usuarios	Registro, inicio de sesion, perfil y cierre de sesion.	Implementado visualmente con datos locales.
Publicaciones	Crear, editar, eliminar, consultar y marcar como vendido.	Implementado con repositorio en memoria.
Categorias	Libros, Electronicos, Laboratorio, Tutorias y Otros.	Implementado con chips y selector Spinner.
Multimedia	Agregar imagenes y video demostrativo.	Simulado mediante selectores de contenido.
Favoritos	Agregar, eliminar y consultar favoritos.	Implementado con conjunto local de favoritos.
Mensajeria	Contactar vendedor, enviar y recibir mensajes.	Implementado como chat simulado.

Tabla 2. Requisitos funcionales principales

4.3 Diseño de la aplicacion Android

El sistema utiliza una sola Activity llamada MainActivity. Esta Activity funciona como contenedor principal de navegacion y aloja el NavHostFragment. Las pantallas no fueron creadas como Activities independientes, sino como Fragments. Esta decision permite un flujo mas ordenado y facilita centralizar la navegacion.

Fragments implementados: LoginFragment, RegisterFragment, HomeFragment, ProductDetailFragment, PublishProductFragment, FavoritesFragment, ChatFragment y ProfileFragment.

4.4 Interfaces del sistema

Pantalla	Archivo XML	Funcion principal
Login	fragmento_inicio_sesion.xml	Autenticar al usuario mediante correo y contrasena.
Registro	fragmento_registro.xml	Registrar nombre, apellido, correo, contrasena, carrera y telefono.
Inicio	fragmento_inicio.xml	Mostrar publicaciones, busqueda, categorias, favoritos, perfil y publicar producto.
Detalle producto	fragmento_detalle_producto.xml	Mostrar informacion completa del producto y contactar vendedor.
Publicar producto	fragmento_publicar_producto.xml	Crear o editar publicaciones con titulo, descripcion, precio, categoria y multimedia.
Favoritos	fragmento_favoritos.xml	Listar publicaciones guardadas y permitir eliminarlas.
Chat	fragmento_chat.xml	Enviar y visualizar mensajes entre comprador y vendedor.
Perfil	fragmento_perfil.xml	Mostrar datos personales, publicaciones propias, ventas y favoritos.

Tabla 3. Interfaces del sistema

4.5 Modelo de datos local

En la version actual se usa el archivo MarketplaceRepository.kt como fuente de datos temporal. En este archivo se definieron constructores Kotlin mediante data class y enum class con nombres en espanol para facilitar la defensa del proyecto.

Modelo	Campos principales	Descripcion
Usuario	nombre, apellido, correo, carrera, telefono	Representa al estudiante registrado en la aplicacion.
Producto	id, titulo, descripcion, precio, categoria, estado, vendedor, multimedia	Representa una publicacion dentro del marketplace.
CategoriaProducto	LIBROS, ELECTRONICOS, LABORATORIO, TUTORIAS, OTROS	Enumera las categorias disponibles.
MensajeChat	remitente, contenido, enviadoPorUsuarioActual	Representa los mensajes enviados en una conversacion.

Tabla 4. Modelo de datos principal

4.6 Eventos y navegacion

Los eventos de la aplicacion se generan mediante listeners de Android, principalmente `setOnClickListener`. Un boton no crea una Activity; el boton dispara un evento y el codigo dentro del listener ejecuta una accion, por ejemplo navegar a otro Fragment, guardar un producto, eliminar un favorito o enviar un mensaje.

Evento	Accion ejecutada	Pantalla relacionada
botonIniciarSesion	Valida campos y navega al inicio.	Login
botonRegistrarse	Navega a la pantalla de registro.	Login
botonPublicarProducto	Navega al formulario de publicacion.	Inicio
botonFavorito	Agrega o elimina una publicacion de favoritos.	Inicio y detalle
botonContactarVendedor	Abre la pantalla de chat del producto.	Detalle producto
botonPublicar	Crea o actualiza una publicacion.	Publicar producto
botonEnviar	Agrega el mensaje al historial del chat.	Chat
botonCerrarSesion	Regresa a la pantalla de inicio de sesion.	Perfil

Tabla 5. Eventos principales de la aplicacion

4.7 Base de datos PostgreSQL

Para la persistencia futura se preparo el archivo `database/unimarket_postgresql.sql`. El script crea las tablas `users`, `categories`, `products`, `product_media`, `favorites`, `conversations` y `messages`. Tambien incluye indices, triggers de actualizacion y datos de prueba. La aplicacion Android no debe conectarse directamente a PostgreSQL; lo recomendable es crear una API REST intermedia para proteger credenciales y controlar la logica de negocio.

4.8 Pruebas realizadas

La aplicacion fue verificada mediante la compilacion del APK debug y la ejecucion de pruebas unitarias locales. Los comandos ejecutados fueron:

- `.\gradlew.bat assembleDebug`
- `.\gradlew.bat testDebugUnitTest`

Ambos procesos finalizaron correctamente, por lo que el proyecto compila y genera el APK debug sin errores.

5. Conclusiones y trabajo futuro

5.1 Conclusiones

El desarrollo de UniMarket permitio construir una base funcional para un marketplace universitario. La aplicacion ya cuenta con pantallas principales, navegacion completa, eventos de botones, datos simulados y una estructura preparada para persistencia real. La organizacion mediante una sola Activity y varios Fragments facilita el mantenimiento del proyecto y permite explicar con claridad la arquitectura durante la defensa.

5.2 Lineas de trabajo futuro

- Implementar una API REST con Node.js o Spring Boot.
- Conectar la API REST con la base de datos PostgreSQL.
- Reemplazar MarketplaceRepository por servicios remotos.
- Agregar autenticacion real con contraseñas cifradas y tokens.
- Subir imagenes y videos a almacenamiento seguro.
- Implementar chat persistente e historial real de conversaciones.
- Agregar validacion de correo institucional.

6. Bibliografia

Android Developers. Documentacion oficial de Android y Kotlin para el desarrollo de aplicaciones moviles.

Google Material Design. Guia de componentes y principios de Material Design 3.

PostgreSQL Global Development Group. Documentacion oficial de PostgreSQL.

Documentacion interna del proyecto UniMarket desarrollada en Android Studio.

Anexos

Anexo A. Archivos principales del proyecto

Los archivos principales del proyecto son MainActivity.kt, MarketplaceRepository.kt, los fragments Kotlin de login, registro, inicio, detalle, publicacion, favoritos, chat y perfil, ademas de los XML fragmento_inicio_sesion.xml, fragmento_registro.xml, fragmento_inicio.xml, fragmento_detalle_producto.xml, fragmento_publicar_producto.xml, fragmento_favoritos.xml, fragmento_chat.xml y fragmento_perfil.xml.

La navegacion se encuentra en grafo_navegacion.xml y la propuesta de persistencia se encuentra en database/unimarket_postgresql.sql.